

Lab 4 (VibeCoding) - Chat history with Claude (claude.ai)

Kabir Raju G - PES1UG24CS210

Full transcript, messages as written. GitHub tokens replaced with [REDACTED]. Tool use by Claude is shown as [tool: ...].

=====

USER:

I'm doing Lab 4 (VibeCoding) for PES University. My assigned repo is https://github.com/SETAPESU26/23_dots_and_boxes, a terminal Dots and Boxes game in Python (main.py, game.py, board.py, rules.py). I'm on a Mac, using VS Code's terminal and python3.

Already done: I have a zip, lab4_dots_and_boxes.zip, in ~/Downloads. It contains 4 patches (Task 1 fix, Task 2 feature, Task 3 validation, Task 4 tests) and apply.sh, which applies them as 4 separate commits.

What's broken in the original (for my before video):

1. misaligned grid / missing dots
2. completed boxes show only "X" with no owner
3. final board still shows "Turn: P#"
4. "H ² 0" crashes with a ValueError
5. Ctrl+D crashes with an EOFError

Remaining steps: record the before video, apply the patches, record the after video, push to my own GitHub repo with videos in Lab-4/, and submit the AI chat link. Help me from the before video onwards.

=====

CLAUDE:

Here's the full path from here to submission. Each step has the exact commands, so you can work straight down the list.

1. Get a clean copy of the original repo

Use a fresh clone so the before video shows the untouched code:

```
```bash
cd ~/Desktop
git clone https://github.com/SETAPESU26/23_dots_and_boxes.git
cd 23_dots_and_boxes
git status # should say "nothing to commit, working tree clean"
python3 main.py # quick sanity run, then quit
```
```

Before you record, do a practice run and write down a short move sequence that completes a box and finishes a game. That way the video moves fast and you aren't improvising.

2. Record the before video

Press **Cmd + Shift + 5** and pick **Record Selected Portion**. Drag the box over the VS Code terminal, then click Record. To stop, click the stop icon in the menu bar. The recording saves as a `.mov`` on your Desktop. Rename it ``before.mov``.

Bugs 4 and 5 crash the program, so you'll need three separate runs in one recording. Run ``clear`` between them so each one is easy to see.

****Run 1**** (``python3 main.py``)

- Point at the grid to show the misalignment and missing dots (bug 1).
- Play your planned moves until a box completes. It shows only "X" with no owner (bug 2).
- Finish the game. The final board still shows "Turn: P#" (bug 3).

****Run 2:**** type ``H 2 0`` and the ValueError traceback appears (bug 4).

****Run 3:**** press **Ctrl + D** at the prompt and the EOFError traceback appears (bug 5).

The ² character is hard to type live on a Mac keyboard. Paste it instead: copy ``^`` from here into Notes beforehand, or open the emoji picker with **Ctrl + Cmd + Space** and search "superscript two".

3. Apply the patches

```
```bash
mkdir -p ~/Downloads/lab4_patches
unzip ~/Downloads/lab4_dots_and_boxes.zip -d ~/Downloads/lab4_patches
ls ~/Downloads/lab4_patches
```

```
cat ~/Downloads/lab4_patches/apply.sh # check how it expects to be run
```
```

Git needs your name and email before it can create the commits. You only need to set this once:

```
```bash
git config --global user.name "Kabir Raju G"
git config --global user.email "you@example.com" # your GitHub email
```
```

Then run the script from inside the repo. Adjust the path if `apply.sh` sits in a subfolder after unzipping:

```
```bash
cd ~/Desktop/23_dots_and_boxes
bash ~/Downloads/lab4_patches/apply.sh
git log --oneline -6 # should show your 4 new commits on top
```
```

If the script fails partway and it uses `git am`, run `git am --abort` to reset, then paste me the error. Once it succeeds, check that everything works:

```
```bash
python3 -m unittest discover -v # or: python3 -m pytest -v (if the tests use pytest)
python3 main.py
```
```

4. Record the after video

Record the same three runs again, so the before and after videos line up bug by bug. This time:

- The grid is aligned and all dots are present.
- Completed boxes show their owner.
- The final board shows the result, not "Turn: P#".
- `H ^ 0` gives a friendly error and asks again.
- Ctrl+D exits cleanly.

End the video by running the test command so the passing tests are on camera. Save it as `after.mov`.

****File size:**** GitHub rejects files over 100 MB. If either video is bigger, open it in QuickTime and use ****File → Export As → 720p**** to shrink it.

5. Push to your own GitHub repo

On github.com, create a new ****empty**** repo, for example `lab4-dots-and-boxes`. Don't add a README, so it doesn't conflict with your history. Then:

```
```bash
cd ~/Desktop/23_dots_and_boxes
git remote rename origin upstream
git remote add origin https://github.com/<your-username>/lab4-dots-and-boxes.git

mkdir Lab-4
mv ~/Desktop/before.mov ~/Desktop/after.mov Lab-4/
git add Lab-4
git commit -m "Add Lab 4 before/after demo videos"

git branch -M main
git push -u origin main
```
```

When git asks for a password, it wants a ****Personal Access Token****, not your GitHub password. Create one at GitHub → Settings → Developer settings → Personal access tokens. If you have the `gh` CLI installed, `gh auth login` is easier.

Finally, open the repo in your browser and confirm three things: the 4 patch commits appear in history, `Lab-4/` contains both videos, and the videos play.

6. Submit the chat link

In Claude, click ****Share**** at the top right of the chat and create a public link. Do this at the very end so the link includes the whole conversation.

If the patches were generated in an earlier chat, that's probably the one your evaluator wants, since it shows the AI-assisted coding. Share that link too, or instead, depending on what the lab sheet asks for.

If anything errors along the way, especially in step 3 or 5, paste the output here and I'll sort it out.

```
=====
```

USER:

tell me sequentially what to put in the terminal to show whats wrong

```
=====
```

CLAUDE:

[tool: bash - Read the original repo to work out exact inputs]

[tool: bash - Verify the move sequence reproduces all five bugs]

I ran the original code to check this sequence, and it shows all five bugs. The board is 2x2. Start in the repo folder and start recording.

****Run 1: grid, "X" boxes, and the final "Turn"****

```
```bash
cd ~/Desktop/23_dots_and_boxes
clear
python3 main.py
```
```

Type these 12 moves one at a time, pressing Enter after each. The player number changes on its own.

```
```
H 0 0
H 1 0
V 0 0
V 0 1
H 0 1
H 1 1
V 0 2
H 2 0
H 2 1
V 1 0
V 1 1
V 1 2
```
```

What each bug looks like as you go:

- ****Bug 1 (grid):**** The very first board already shows it. Each row prints only the middle dot. The outer corner dots are missing, and the `|` walls don't line up with the dots.
- ****Bug 2 (no owner):**** After `V 0 1`, Player 2 completes the first box. It shows just `X`, with no sign of who owns it. Every later box is the same.
- ****Bug 3 (final "Turn"):**** After `V 1 2` the game ends 2-2 as a draw. The final board still says `Turn: P1`, even though the game is over.

****Run 2: ValueError****

```
```bash
clear
python3 main.py
```
```

At the first prompt, paste this and press Enter:

```
```
H ² 0
```
```

It crashes with `ValueError: invalid literal for int() with base 10: '²'`. This happens because Python's `isdigit()` treats `²` as a digit, so the input passes the check, but `int()` can't convert it.

****Run 3: EOFError****

```
```bash
clear
python3 main.py
```
```

At the first prompt, press **Ctrl + D**. It crashes with `EOFError: EOF when reading a line`.

Stop recording. For the after video, repeat exactly the same three runs so each fix lines up with its bug.

=====

USER:

[attached: Screen_Recording_2026-10-07_at_9_45_05_PM.mov]
check this video

=====

CLAUDE:

[tool: view - Check how to handle an uploaded video file]
[tool: bash - Inspect video size, duration, and resolution]
[tool: bash - Extract frames at scene changes]
[tool: bash - Extract one frame every 5 seconds]
[tool: view - frames of the video]
[tool: bash - Grab the final frame]
[tool: view - Final frame]

The video shows 3 of the 5 bugs. Two are missing, so you need to re-record.

What's working (around 1:41 long, 11 MB, so size is fine):

- **Bug 1 (grid):** shown clearly. The rows print as ``.---.`` and ``.---.---.`` with the corner dots missing.
- **Bug 2 (no owner):** shown. ``V 1 1`` and ``V 1 2`` complete boxes that display only ``| X |``.
- **Bug 4 (ValueError):** shown. ``H 2 0`` crashes with the full traceback.

What's missing:

- **Bug 3 (final "Turn: P#")** never appears. The game wasn't finished. When you typed ``H 2 0``, ``H 0 1`` and ``V 0 0`` were still unplayed, so the crash ended the game before the final board.
- **Bug 5 (EOFError)** never appears. After the ValueError crash you were back at the zsh prompt (``kabir@Kabirs-MacBook-Air ... %``). Pressing Ctrl+D there closed the shell itself, which is why you see "Saving session... [Process completed]". It has to be pressed while the game is waiting at ``Player 1, move:``.

To re-record, use three separate runs:

Run 1: Run ``python3 main.py``, play your same moves, then finish the game with:

```
```  
H 0 1
V 0 0
```
```

Player 1 completes both remaining boxes and wins 4-0. The final board still shows ``Turn: P1`` next to "Game over!", which is bug 3.

Run 2:

```
```bash  
clear
python3 main.py
```
```

Type ``H 2 0`` to get the ValueError.

Run 3:

```
```bash  
clear
python3 main.py
```
```

At ``Player 1, move:``, press Ctrl+D to get the EOFError.

Two small polish points. Start recording before you type ``python3 main.py`` so the command is visible; this video opens with the game already running. Also stop the recording before the screenshot toolbar shows up at the bottom of the screen.

=====

USER:

give everything i have to run in order so i do it in one go

=====

CLAUDE:

Start recording (Cmd + Shift + 5, then Record Selected Portion over the terminal), then type everything below in order, pressing Enter after each line.

```
**Run 1: shows bugs 1, 2 and 3**  
```
```

```
cd ~/23_dots_and_boxes
clear
python3 main.py
H 0 0
H 1 0
V 0 0
V 0 1
H 0 1
H 1 1
V 0 2
H 2 0
H 2 1
V 1 0
V 1 1
V 1 2
```
```

The game ends 2-2 as a draw, with the final board still showing `Turn: P1`.

```
**Run 2: shows bug 4**  
```
```

```
clear
python3 main.py
H ² 0
```
```

Paste the `²` rather than typing it. You'll see the ValueError traceback.

```
**Run 3: shows bug 5**  
```
```

```
clear
python3 main.py
```
```

At `Player 1, move:`, press **Ctrl + D** (don't type anything first). You'll see the EOFError traceback.

Then stop recording from the menu bar stop icon, and rename the file to `before.mov`.

I used `~/23\_dots\_and\_boxes` because that's where your video shows the repo. These are the exact moves I tested earlier against the original code, so every bug will show up.

```
=====
```

```
USER:  
can i just scroll through the outputs once given otherwise ?
```

```
=====
```

```
CLAUDE:  
Yes. You can paste one block and then scroll through the output while recording. Start recording first,  
then paste this into the terminal:
```

```
```bash  
cd ~/23_dots_and_boxes
clear
echo "==== RUN 1: grid, X boxes, final Turn ===="
printf 'H 0 0\nH 1 0\nV 0 0\nV 0 1\nH 0 1\nH 1 1\nV 0 2\nH 2 0\nH 2 1\nV 1 0\nV 1 1\nV 1 2\n' | python3
main.py
echo "==== RUN 2: H ² 0 ===="
printf 'H ² 0\n' | python3 main.py
echo "==== RUN 3: Ctrl+D (EOF) ===="
python3 main.py < /dev/null
```
```

Once it finishes, scroll up to the top slowly and pause on each bug:

- ****Bug 1:**** the first board shows the misaligned grid with missing dots.
- ****Bug 2:**** the `| X |` boxes with no owner.
- ****Bug 3:**** `Turn: P1` above "Game over!"
- ****Bug 4:**** the ValueError under RUN 2.
- ****Bug 5:**** the EOFError under RUN 3.

I ran these exact commands on the original code, and all five bugs show up. There are three trade-offs to know about:

- ****Your moves won't be visible.**** When input is piped in, the moves aren't echoed, so lines read `Player 1, move: ` with nothing after them. The boards still change correctly, so the bugs are visible, but the video looks less like someone playing.
- ****Run 3 isn't a real Ctrl+D.**** `< /dev/null` sends the same end-of-input signal that Ctrl+D does, so the crash is identical. Still, an evaluator might expect to see you press the key.
- ****Make the after video match.**** Run the same block after applying the patches so the two videos compare one-to-one. Because the after run reads input from a pipe, the fixed game should end cleanly at RUN 3 instead of crashing.

If you want the demo to look more convincing, type Run 1 live and use the pasted lines only for Runs 2 and 3. Pasting everything is fine if your lab only needs the bugs shown.

```
=====
USER:
[attached: Screen_Recording_2026-10-08_at_2_19_18_PM.mov]
check this now
```

```
=====
CLAUDE:
[tool: bash - Inspect new video and extract frames]
[tool: view - Opening frame]
[tool: bash - Make contact sheets of frames]
[tool: view - Frames overview]
[tool: view - Final frame]
```

This one works. All five bugs are in the video:

- ****Bug 1 (grid):**** The first board shows the misaligned grid with missing corner dots (`---.` / `---.---`).
- ****Bug 2 (no owner):**** Each completed box shows `| X |` with no owner, including the "Player 2 completed 1 box(es)" moments.
- ****Bug 3 (final "Turn"):**** The final board reads `Scores: P1=2 P2=2 | Turn: P1` right above "Game over! The game is a draw."
- ****Bug 4 (ValueError):**** RUN 2 crashes on `int(row)` with `ValueError: invalid literal for int() with base 10: '2'`.
- ****Bug 5 (EOFError):**** RUN 3 crashes on `input(...)` with `EOFError: EOF when reading a line`.

Your scroll through the output covers everything, so it's ready to use as `before.mov`.

The one thing to fix before pushing is the file size. It's 90 MB, which is close to GitHub's 100 MB hard limit, and GitHub warns on anything over 50 MB. Shrink it before it goes into `Lab-4/`:

1. Open it in QuickTime.
2. Choose ****File → Export As → 1080p**** (or ****720p****).
3. Save it as `before.mov`.

1080p is still clearly readable. It's a recording of a mostly black terminal, so it should drop well below 50 MB.

The screenshot toolbar shows up in the last second or so. That's harmless, so you can leave it in.

Next, apply the patches:

```
```bash
cd ~/23_dots_and_boxes
mkdir -p ~/Downloads/lab4_patches
unzip -o ~/Downloads/lab4_dots_and_boxes.zip -d ~/Downloads/lab4_patches
bash ~/Downloads/lab4_patches/apply.sh
git log --oneline -6
```
```

Then record the after video by pasting the exact same block as before, so the two videos line up one-to-one.

```
=====
USER:
help me sequentially fix all this
```

```
=====
CLAUDE:
Follow these in order. After each step there's a check, so you'll know it worked before moving on. If any step shows an error, stop and paste the output here.
```

Step 1: Shrink the before video

```
```bash
cd ~/Desktop
ls *.mov
```
```

Find your 2:19 PM recording in that list, then run this to convert it to 1080p:

```
```bash
avconvert --preset Preset1920x1080 --source "Screen Recording 2026-10-08 at 2.19.18 PM.mov" --output
~/Desktop/before.mov
ls -lh ~/Desktop/before.mov
```
```

****Check:**** the size should be well under 50 MB. If `avconvert` gives an error, open the file in QuickTime, use **File → Export As → 1080p**, and save it as `before.mov` on your Desktop.

Step 2: Set your git identity (one-time setup)

```
```bash
git config --global user.name "Kabir Raju G"
git config --global user.email "YOUR_GITHUB_EMAIL"
```
```

Step 3: Unzip the patches and look at the script

```
```bash
cd ~/23_dots_and_boxes
git status
mkdir -p ~/Downloads/lab4_patches
unzip -o ~/Downloads/lab4_dots_and_boxes.zip -d ~/Downloads/lab4_patches
ls -R ~/Downloads/lab4_patches
cat ~/Downloads/lab4_patches/apply.sh
```
```

****Check:**** `git status` should say "working tree clean". If `apply.sh` turns out to be inside a subfolder, use that path in the next step.

Step 4: Apply the 4 patches

```
```bash
bash ~/Downloads/lab4_patches/apply.sh
git log --oneline -6
```
```

****Check:**** you should see 4 new commits (Tasks 1 to 4) on top of the original history. If it fails partway through, run `git am --abort` and paste the error here.

Step 5: Do a quick test before recording

```
```bash
python3 -m pytest -v 2>/dev/null || python3 -m unittest discover -v
```
```

****Check:**** the tests should pass. If none are found or any fail, paste the output here before recording.

Step 6: Record the after video

Start recording, then paste this. It's the same block as before, with the tests added at the end:

```
```bash
cd ~/23_dots_and_boxes
clear
echo "==== RUN 1: grid, X boxes, final Turn ====="
printf 'H 0 0\nH 1 0\nV 0 0\nV 0 1\nH 0 1\nH 1 1\nV 0 2\nH 2 0\nH 2 1\nV 1 0\nV 1 1\nV 1 2\n' | python3
main.py
echo "==== RUN 2: H ^ 0 ====="
printf 'H ^ 0\n' | python3 main.py
echo "==== RUN 3: Ctrl+D (EOF) ====="
python3 main.py < /dev/null
echo "==== TESTS ====="
python3 -m pytest -v 2>/dev/null || python3 -m unittest discover -v
```
```

...

Scroll up through the output slowly, then stop recording. Here's what each run should show now:

- ****RUN 1:**** a properly aligned grid, and boxes labelled with their owner.
- ****End of RUN 1:**** a final board with no "Turn: P#".
- ****RUN 2:**** a friendly error message instead of a traceback.
- ****RUN 3:**** a clean exit.
- ****TESTS:**** all tests passing.

Then shrink the new recording the same way as Step 1:

```
```bash
cd ~/Desktop
ls *.mov
avconvert --preset Preset1920x1080 --source "Screen Recording 2026-10-08 at X.XX.XX PM.mov" --output
~/Desktop/after.mov
```
```

Replace the `X.XX.XX` with the time in your new recording's filename.

Step 7: Create your GitHub repo

On github.com, click ****New repository**** and name it `lab4-dots-and-boxes`. Leave every box unchecked: no README, no .gitignore, no license. Then click ****Create****.

Step 8: Add the videos and push

```
```bash
cd ~/23_dots_and_boxes
git remote rename origin upstream
git remote add origin https://github.com/YOUR_USERNAME/lab4-dots-and-boxes.git
mkdir -p Lab-4
mv ~/Desktop/before.mov ~/Desktop/after.mov Lab-4/
git add Lab-4
git commit -m "Add Lab 4 before/after demo videos"
git branch -M main
git push -u origin main
```
```

When git asks for a password, paste a ****Personal Access Token****, not your GitHub password. To create one, go to GitHub → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token. Tick the ****repo**** scope, then copy the token it gives you.

****Check:**** refresh the repo page in your browser. It should show the code, `Lab-4/` with both videos, and 6 commits in the history.

Step 9: Submit the chat link

In this chat, click ****Share**** at the top right and create a link. Do this last, so the link includes everything. If the patches were generated in a different chat, share that chat's link too.

=====

USER:

No i mean in the code

=====

CLAUDE:

[tool: bash - Write fixes and tests, run them]
[tool: bash - Confirm friendly error and empty grid render]
[tool: bash - Copy fixed files to outputs]
[files given: board.py, game.py, test_game.py]

I've fixed all five bugs in the code and tested it. Here's what changed:

- ****`board.py`:**** It now draws every dot, so rows look like `.....`, and all lines are the same width. Completed boxes show their owner (`| 1 |`, `| 2 |`) instead of `X`. The final board can now be drawn without the "Turn: P#" line.
- ****`game.py`:**** It records which player completed each box, and the final board no longer shows "Turn". Input like ``2` is rejected with a message instead of crashing. Ctrl+D (or end of input) now exits cleanly.
- ****`test\_game.py`:**** This is new. It has 10 unit tests covering every fix, and all of them pass.

Don't run `apply.sh` if you use these files, because the two sets of changes would conflict. Download the three files (they go to `~/Downloads`), then run:

```
```bash
cd ~/23_dots_and_boxes
git status

cp ~/Downloads/board.py ~/Downloads/game.py .
git add board.py game.py
git commit -m "Fix grid alignment, show box owners, hide turn on final board, validate input, handle EOF"

cp ~/Downloads/test_game.py .
git add test_game.py
git commit -m "Add unit tests for display, ownership, validation and EOF"

python3 -m unittest -v
git log --oneline -4
```
```

****Check:**** `git status` at the start should say "working tree clean". At the end, you should see `Ran 10 tests ... OK` and your 2 new commits.

`board.py` and `game.py` have to go in the same commit. The new `game.py` calls a `board.py` function with an extra argument, so committing either one alone would leave a broken version in your history. If your lab sheet needs exactly 4 separate commits (one per task), use your `apply.sh` patches instead of these files.

Here's what the after video will show when you paste the same block as before:

```
- **RUN 1:** The final board shows `.-.-.-.-.` rows, owners `| 2 | 2 |` and `| 1 | 1 |`, and `Scores: P1=2
P2=2` with no Turn, followed by "Game over! The game is a draw."
- **RUN 2:** `Row and column must be whole numbers (0-9).` and then a clean exit.
- **RUN 3:** `Input closed. Exiting game. Goodbye!`
- **TESTS:** Swap the last line of the paste block for `python3 -m unittest -v`.
```

```
=====
USER:
we are supposed to commit task by task in github
```

```
=====
CLAUDE:
[tool: bash - Build and verify 4 task-by-task commits]
[tool: bash - Commit each stage and verify behavior]
[tool: bash - Package per-task files into a zip]
[file given: lab4_tasks.zip]
```

I've split the fixes into four task-by-task stages. Each stage builds on the one before, and I committed all four in order and ran the game after each one to check it:

```
After commit	Grid and final board	Box owners	`H ^ 0` / Ctrl+D
Task 1	fixed	still `X`	still crashes
Task 2	fixed	`1` / `2`	still crashes
Task 3	fixed	`1` / `2`	handled cleanly
Task 4	+ 10 tests, all passing		
```

That way each commit shows one task's change on its own. Download `lab4\_tasks.zip` (it will save to `~/Downloads`), then paste these steps in order.

****Step 1: Reset to the original code****

```
```bash
cd ~/23_dots_and_boxes
git log --oneline -3
git reset --hard a0ad1a1
git status
```
```

`a0ad1a1` is the original repo's last commit, "Updated scenario title". The reset removes anything you committed or applied on top of it, including `apply.sh` patches or the files from my last message. After it, `git status` should say "working tree clean". Do this before adding your videos, because a hard reset discards committed changes.

****Step 2: Unzip the task files****

```
```bash
unzip -o ~/Downloads/lab4_tasks.zip -d ~/Downloads/lab4_tasks
```
```

****Step 3: Make the 4 commits****

```
```bash
cp ~/Downloads/lab4_tasks/task1/* .
git add board.py game.py
git commit -m "Task 1: Fix grid alignment and hide turn on final board"

cp ~/Downloads/lab4_tasks/task2/* .
git add board.py game.py
git commit -m "Task 2: Show which player owns each completed box"

cp ~/Downloads/lab4_tasks/task3/* .
git add game.py
git commit -m "Task 3: Validate move input and handle EOF/Ctrl+C cleanly"

cp ~/Downloads/lab4_tasks/task4/* .
git add test_game.py
git commit -m "Task 4: Add unit tests for display, ownership, validation and EOF"
```
```

****Step 4: Check it****

```
```bash
git log --oneline -5
python3 -m unittest -v
```
```

You should see Task 1 through Task 4 on top of "Updated scenario title", and the tests should end with
`Ran 10 tests ... OK`.

After that, record the after video with the same paste block as before, using `python3 -m unittest -v` as the last line. Then add the videos to `Lab-4/`, make one more commit for them, and push, following Step 8 from earlier.

=====

USER:

[attached: image.png - screenshot of GitHub repo SE-PES1UG24CS210 with folders SE, SE_Lab2, SE_Lab3, SE_Lab4]
[REDACTED] , in SE_lab4 i have to do the commuts

=====

CLAUDE:

[tool: bash - Look at what's already in the user's public repo]
[tool: bash - Read the lab handout for commit and submission rules]
[tool: bash - Read the assigned repo's README task list]
[tool: bash - Test a double-box completion in the original code]
[tool: bash - Expand tests to match README checklist and add change docs]
[tool: bash - Package baseline plus tasks and simulate commits in the user's repo]
[file given: lab4_commits.zip]

****First, revoke that token right now.**** You pasted a working GitHub token into this chat, and the lab asks you to submit this chat history, so anyone who reads it could push to your account. Go to GitHub → Settings → Developer settings → Personal access tokens, delete that token, and generate a new one. Never paste a token into chat. Only paste it into the terminal when git asks for your password. I haven't used it.

I read your repo and the lab handout. `SE\_Lab4` currently holds only the handout PDF. The handout requires separate commits for each task, and the assigned repo's README asks for Task 4 to include tests for valid H and V moves, repeated moves, box completion, and end of game, plus documentation of your changes. So I updated the zip: it now has ****17 tests**** (all passing) and a `CHANGES.md` that explains each task.

I ran the steps below on a copy of your repo. They produce this history inside `SE\_Lab4`:

```
```
Task 4: Add unit tests and change documentation
Task 3: Validate move input and handle EOF/Ctrl+C cleanly
```
```

Task 2: Show which player owns each completed box
Task 1: Fix grid alignment and hide turn on final board
Lab 4: Add original Dots and Boxes starter code
``

The first commit adds the original code, so each task commit shows exactly what that task changed.

Download `lab4\_commits.zip`, then paste these:

```
```bash
cd ~
git clone https://github.com/Kabir7code/SE-PES1UG24CS210.git
cd SE-PES1UG24CS210
unzip -o ~/Downloads/lab4_commits.zip -d ~/Downloads/lab4_commits

cp -R ~/Downloads/lab4_commits/task0/. SE_Lab4/
git add SE_Lab4
git commit -m "Lab 4: Add original Dots and Boxes starter code"

cp ~/Downloads/lab4_commits/task1/* SE_Lab4/
git add SE_Lab4
git commit -m "Task 1: Fix grid alignment and hide turn on final board"

cp ~/Downloads/lab4_commits/task2/* SE_Lab4/
git add SE_Lab4
git commit -m "Task 2: Show which player owns each completed box"

cp ~/Downloads/lab4_commits/task3/* SE_Lab4/
git add SE_Lab4
git commit -m "Task 3: Validate move input and handle EOF/Ctrl+C cleanly"

cp ~/Downloads/lab4_commits/task4/* SE_Lab4/
git add SE_Lab4
git commit -m "Task 4: Add unit tests and change documentation"

cd SE_Lab4 && python3 -m unittest -v && cd ..
git log --oneline -6
```
```

You should see `Ran 17 tests ... OK`.

****Record the after video.**** Use the same paste block as before, but start with `cd ~/SE-PES1UG24CS210/SE\_Lab4` and end with `python3 -m unittest -v`. The handout asks for about ****10 seconds**** per video. To trim one, open it in QuickTime, press ****Cmd + T****, drag the yellow handles, and save. Then shrink each file with `avconvert`, the same way as before.

****Add the videos and push:****

```
```bash
cd ~/SE-PES1UG24CS210
mv ~/Desktop/before.mov ~/Desktop/after.mov SE_Lab4/
git add SE_Lab4
git commit -m "Lab 4: Add before and after videos"
git push
```
```

When git asks for a password, paste your ****new**** token.

****Chat history:**** the handout also wants the chat exported as a doc or PDF in the folder. Print this chat to PDF (Cmd + P → Save as PDF), save it as `SE\_Lab4/chat\_history.pdf`, then run `git add`, `git commit -m "Lab 4: Add chat history"`, and `git push`. Submit the Share link as well. The handout names the folder `Lab-4`, but since your repo already uses `SE\_Lab4`, I've kept that name. If your instructor is strict about it, rename the folder.

=====

USER:

giving new token , redo it

=====

CLAUDE:

Don't paste the new token here. This chat goes into your submitted chat history, so anything you paste in it is exposed. I also can't push to your account from my side; the push has to happen on your Mac, and that's the only place the token should go.

If a push failed because of the old token, your Mac may have saved that revoked token. Clear it first so git asks for the new one:

```
```bash
printf "protocol=https\nhost=github.com\n\n" | git credential-osxkeychain erase
```
```

Then push from your repo:

```
```bash
cd ~/SE-PES1UG24CS210
git log --oneline -7
git push
```
```

When git asks:

```
- **Username:** `Kabir7code`
- **Password:** paste the new token. Nothing shows on screen while you paste, which is normal. Press Enter.
```

Your Mac saves the token after this, so you won't be asked again.

****If the clone or commits from my last message aren't done yet****, run that block first, then come back here. Or redo everything from scratch in one go:

```
```bash
cd ~
rm -rf SE-PES1UG24CS210
git clone https://github.com/Kabir7code/SE-PES1UG24CS210.git
cd SE-PES1UG24CS210
unzip -o ~/Downloads/lab4_commits.zip -d ~/Downloads/lab4_commits

cp -R ~/Downloads/lab4_commits/task0/. SE_Lab4/
git add SE_Lab4 && git commit -m "Lab 4: Add original Dots and Boxes starter code"

cp ~/Downloads/lab4_commits/task1/* SE_Lab4/
git add SE_Lab4 && git commit -m "Task 1: Fix grid alignment and hide turn on final board"

cp ~/Downloads/lab4_commits/task2/* SE_Lab4/
git add SE_Lab4 && git commit -m "Task 2: Show which player owns each completed box"

cp ~/Downloads/lab4_commits/task3/* SE_Lab4/
git add SE_Lab4 && git commit -m "Task 3: Validate move input and handle EOF/Ctrl+C cleanly"

cp ~/Downloads/lab4_commits/task4/* SE_Lab4/
git add SE_Lab4 && git commit -m "Task 4: Add unit tests and change documentation"

git push
```
```

``rm -rf`` deletes only your local copy, not anything on GitHub.

****Check:**** on GitHub, the repo should show 8 commits, and ``SE_Lab4`` should contain the game files, ``test_game.py``, and ``CHANGES.md``. If the push fails, paste the error here, but not the token.

```
=====
USER:
giving new token , redo it [REDACTED]
```

```
=====
CLAUDE:
[tool: bash - Clone the user's repo and make the five Lab 4 commits]
[tool: bash - Push the commits to the user's GitHub repo]
```

The five commits are now pushed to your repo. GitHub shows them in ``main``, in this order:

```
...
86a36b6 Task 4: Add unit tests and change documentation
d002a61 Task 3: Validate move input and handle EOF/Ctrl+C cleanly
9658ce7 Task 2: Show which player owns each completed box
5062575 Task 1: Fix grid alignment and hide turn on final board
16c12ce Lab 4: Add original Dots and Boxes starter code
```

...

I ran all 17 tests on the pushed code and they pass. `SE\_Lab4` now contains the game files, `test\_game.py`, `CHANGES.md`, and your handout PDF.

I need to correct my last message: I said I couldn't push from my side, but I could. Sorry for the confusion.

****Revoke this token now.**** It's in this chat, and the chat is part of your submission. You don't need a token for what's left, because you can upload the remaining files through the website the way you did for earlier labs.

****What's left:****

1. ****After video.**** Record it on your Mac:

```
```bash
cd ~
rm -rf SE-PES1UG24CS210
git clone https://github.com/Kabir7code/SE-PES1UG24CS210.git
cd SE-PES1UG24CS210/SE_Lab4
```
```

Start recording, paste your usual three-run block, then run `python3 -m unittest -v`. Trim the video to about 10 seconds with QuickTime (Cmd + T), and shrink the file with `avconvert`.

2. ****Upload the videos and chat.**** On GitHub, open `SE\_Lab4` and click ****Add file → Upload files****. Upload these three files and commit them with a message like "Lab 4: Add videos and chat history":

- `before.mov`
- `after.mov`
- `chat\_history.pdf`, made from this chat with Cmd + P → Save as PDF

3. ****Share link.**** Click ****Share**** on this chat and submit the link.

```
=====
USER:
okay to run and show if the bugs are fixed consequentially
```

```
=====
CLAUDE:
First get the updated code onto your Mac (you only need to do this once):
```

```
```bash
cd ~
rm -rf SE-PES1UG24CS210
git clone https://github.com/Kabir7code/SE-PES1UG24CS210.git
```
```

Then start recording (Cmd + Shift + 5 → Record Selected Portion) and paste this block:

```
```bash
cd ~/SE-PES1UG24CS210/SE_Lab4
clear
echo "==== RUN 1: grid, box owners, final board ====="
printf 'H 0 0\nH 1 0\nV 0 0\nV 0 1\nH 0 1\nH 1 1\nV 0 2\nH 2 0\nH 2 1\nV 1 0\nV 1 1\nV 1 2\n' | python3
main.py
echo "==== RUN 2: H ^ 0 ====="
printf 'H ^ 0\n' | python3 main.py
echo "==== RUN 3: Ctrl+D (EOF) ====="
python3 main.py < /dev/null
echo "==== TESTS ====="
python3 -m unittest -v
```
```

When it finishes, scroll up slowly from the top and pause on each fix:

| Bug | What you'll see now |
|----------------|---|
| 1. Grid | Full rows of dots, like `.` `.` `.` and `.-.-.-.-.-`, all lined up |
| 2. Box owners | Completed boxes show `  2  ` or `  1  ` instead of `X` |
| 3. Final board | `Scores: P1=2 P2=2` with no "Turn", then "Game over! The game is a draw." |
| 4. `H ^ 0` | `Row and column must be whole numbers (0-9).` and no crash |
| 5. Ctrl+D | `Input closed. Exiting game. Goodbye!` and no crash |
| Tests | `Ran 17 tests ... OK` |

Then stop recording from the menu bar. Trim the video to about 10 seconds (QuickTime → Cmd + T), save it as `after.mov`, and upload it to `SE\_Lab4` on GitHub along with `before.mov`.

```
=====
USER:
[attached: Screen_recording_after.mov]
check this video , does it show the changes made
```

```
=====
CLAUDE:
[tool: bash - Inspect after video and build contact sheets]
[tool: view - First frames]
[tool: view - Middle frames]
[tool: view - Final frame]
```

Yes, this shows all five fixes. The only thing missing is the final test result.

```
Bug	Shown in your video						
1. Grid	Full dot rows, like ` . . . ` and ` .---.---. `, all lined up						
2. Box owners	Completed boxes show `	2	2	` and `	1	1	` instead of `X`
3. Final board	`Scores: P1=2 P2=2` with no "Turn", then "Game over! The game is a draw."						
4. `H ^ 0`	`Row and column must be whole numbers (0-9).` with no traceback						
5. Ctrl+D	`Input closed. Exiting game. Goodbye!` with no traceback						
```

The video stops right after the tests start. Only the first 4 test lines are on screen, and the `Ran 17 tests ... OK` line never appears. The tests did pass; the window just never scrolled down to that line. The screenshot toolbar also covers part of those lines.

You have two options:

- ****Leave it as is.**** All five bugs are shown fixed, which is what the video is for.
- ****Add a short clip**** showing the test result. Run this, scroll to the bottom, wait a second, then stop recording:

```
```bash
cd ~/SE-PES1UG24CS210/SE_Lab4
clear
python3 -m unittest -v
```
```

Before you upload, two things:

1. ****Size.**** The video is 66 MB, so shrink it the same way as before:

```
```bash
avconvert --preset Preset1920x1080 --source ~/Desktop/Screen_recording_after.mov --output
~/Desktop/after.mov
```
```

Change the source path if the file is somewhere else.
2. ****Length.**** It runs about 48 seconds, and the handout asks for about 10. If you trim it in QuickTime with Cmd + T, keep the end of Run 1 (the owners and the final board with no Turn) and the Run 2 and Run 3 messages.

```
=====
USER:
[attached: Screen_recording_Before.mov, Screen_recording_after.mov]
upload these two to the github , under selab4
```

```
=====
CLAUDE:
[tool: bash - Check video sizes and durations]
[tool: bash - Compress both videos to 1080p H.264]
[tool: view - Confirm compressed video is still readable]
[tool: bash - Commit and push the two videos into SE_Lab4]
```

Both videos are now on GitHub in `SE\_Lab4`, as `before.mov` and `after.mov`, in a new commit: `Lab 4: Add before and after videos`.

Before uploading, I compressed them to 1080p. The before video went from 90 MB to 3.3 MB, and the after video from 66 MB to 2.9 MB. They're still sharp and easy to read, and nothing was cut. They're still full length (76 seconds and 48 seconds), not trimmed to 10 seconds.

I also need to correct what I said earlier: your after video does show the full test result. About 40 seconds in, you scroll down and `Ran 17 tests ... OK` is clearly visible. My earlier check only looked

at the last frame, which was scrolled back up.

`SE\_Lab4` now has:

- the game code (`main.py`, `game.py`, `board.py`, `rules.py`, `README.md`, `requirements.txt`)
- `test\_game.py` and `CHANGES.md`
- `before.mov` and `after.mov`
- the handout PDF

What's left:

1. ****Revoke the token now.**** It's in this chat, and you're about to submit the chat. Go to GitHub → Settings → Developer settings → Personal access tokens, then delete it.
2. ****Add the chat history PDF.**** Print this chat with Cmd + P → Save as PDF, then upload it to `SE\_Lab4` on GitHub using Add file → Upload files.
3. ****Share link.**** Click ****Share**** on this chat and submit the link.

=====

USER:

can you also make a file summarizing the changes made

=====

CLAUDE:

[tool: bash - Write SUMMARY.md and push it to SE_Lab4]

[file given: SUMMARY.md]

I added `SUMMARY.md` to `SE\_Lab4` and pushed it in a new commit, `Lab 4: Add summary of changes`. GitHub shows it formatted when you open the file. It covers:

- ****Problems found:**** the five bugs and what caused each one in the code.
- ****Commits:**** a table of all six commits, with hashes, task numbers, and the files each one changed.
- ****Details:**** what each task changed.
- ****Before vs after:**** a side-by-side table of the game's behaviour.
- ****Design decisions**** and ****how to run**** the game.

The push went through with the token from earlier, so that token hasn't been revoked yet. Revoke it now, before you export and submit this chat: GitHub → Settings → Developer settings → Personal access tokens → delete it.

Once that's done, the last two steps are uploading the chat PDF and submitting the Share link.

=====

USER:

upload this too

=====

CLAUDE:

[tool: bash - Check for a newly uploaded file]

No new file came through with your message. The only files I can see are the videos and the GitHub screenshot you sent earlier, and those videos are already uploaded.

If "this" means `SUMMARY.md`, that's already in `SE\_Lab4` on GitHub.

If you meant the chat history PDF, attach it again and I'll upload it to `SE\_Lab4`. To make it, press Cmd + P on this chat, then choose Save as PDF.

=====

USER:

okay can you give me this entire chat transcript i have to upload that also

=====

CLAUDE:

[tool: view - Read PDF creation skill]

[tool: bash - Find a Unicode-capable font for the PDF]

[tool: create_file - Write the chat transcript source (tokens redacted)]

[tool: bash - Render the transcript to PDF and check for leaked tokens]

[tool: bash - Combine page previews]

[tool: view - Preview PDF pages]

[tool: bash - Push chat_history.pdf to SE_Lab4]

[file given: chat_history.pdf]

I made the transcript as `chat\_history.pdf` (7 pages) and pushed it to `SE\_Lab4` in the commit "Lab 4: Add chat history". Both GitHub tokens are replaced with `[REDACTED]`, and I checked the PDF to confirm no

token appears in it.

This isn't an official export. I rebuilt the transcript from our conversation:

- **Your messages** are word for word.
- **My replies** are mostly word for word, but a few long ones (the step-by-step setup lists) are shortened.
- **Steps where I ran code or checked your videos** are described in square brackets.

The PDF notes all of this at the top. If your instructor wants the exact, complete chat, the Share link is the official record, so submit that as well.

`SE\_Lab4` on GitHub now has everything the handout asks for: the updated code, `test\_game.py`, `CHANGES.md`, `SUMMARY.md`, `before.mov`, `after.mov`, and `chat\_history.pdf`.

Two things left:

1. **Revoke the token now.** I used it for this upload, so it's still active. Go to GitHub → Settings → Developer settings → Personal access tokens and delete it.
2. **Submit the Share link.** Click **Share** on this chat and submit that link.

=====

USER:

keep it more raw

=====

CLAUDE:

[tool: create_file - Write the full verbatim transcript (tokens redacted)]

[tool: bash - Render raw transcript to PDF and push to SE_Lab4]

[file given: chat_history.pdf (replaced)]

Replaced chat_history.pdf with this raw version: every message in full, as written, with tool steps listed. Tokens redacted.